

Research Notes on Masked language model

Masked language model

>> Section 1

GLUE (General Language Understanding Evaluation) task set (consisting of 9 tasks); SQuAD (Stanford Question Answering Dataset) v1.1 and v2.0; SWAG (Situations With Adversarial Generations). In the original paper, all parameters of BERT are fine-tuned, and recommended that, for downstream applications that are text classifications, the output token at the [CLS] input token is fed into a linear-softmax layer to produce the label outputs. The original code base defined the final linear layer as a "pooler layer", in analogy with global pooling in computer vision, even though it simply discards all output tokens except the one corresponding to [CLS] .

>> Section 2

The three attention matrices are added together element-wise, then passed through a softmax layer and multiplied by a projection matrix. Absolute position encoding is included in the final self-attention layer as additional input.

>> Section 3

BERT is meant as a general pretrained model for various applications in natural language processing. That is, after pre-training, BERT can be fine-tuned with fewer resources on smaller datasets to optimize its performance on specific tasks such as natural language inference and text classification, and sequence-to-sequence-based language generation tasks such as question answering and conversational response generation. The original BERT paper published results demonstrating that a small amount of finetuning (for BERTLARGE, 1 hour on 1 Cloud TPU) allowed it to achieved state-of-the-art performance on a number of natural language understanding tasks:

>> Section 4

Tokenizer: This module converts a piece of English text into a sequence of integers ("tokens"). Embedding: This module converts the sequence of tokens into an array of real-valued vectors representing the tokens. It represents the conversion of discrete token types into a lower-dimensional Euclidean space. Encoder: a stack of Transformer blocks with self-attention, but without causal masking. Task head: This module converts the final representation vectors into one-shot encoded tokens again by producing a predicted probability distribution over the token types. It can be viewed as a simple decoder, decoding the latent representation into token types, or as an "un-embedding layer". The task head is necessary for pre-training, but it is often unnecessary for so-called "downstream tasks," such as question answering or sentiment classification. Instead, one removes the task

head and replaces it with a newly initialized module suited for the task, and finetune the new module. The latent vector representation of the model is directly fed into this new module, allowing for sample-efficient transfer learning.

>> Section 5

with probability 80%, the chosen token is masked, resulting in "my1 dog2 is3 [MASK]4"; with probability 10%, the chosen token is replaced by a uniformly sampled random token, such as "happy", resulting in "my1 dog2 is3 happy4"; with probability 10%, nothing is done, resulting in "my1 dog2 is3 cute4". After processing the input text, the model's 4th output vector is passed to its decoder layer, which outputs a probability distribution over its 30,000-dimensional vocabulary space.

>> Section 6

Interpretation Language models like ELMo, GPT-2, and BERT, spawned the study of "BERTology", which attempts to interpret what is learned by these models. Their performance on these natural language understanding tasks are not yet well understood. Several research publications in 2018 and 2019 focused on investigating the relationship behind BERT's output as a result of carefully chosen input sequences, analysis of internal vector representations through probing classifiers, and the relationships represented by attention weights. The high performance of the BERT model could also be attributed to the fact that it is bidirectionally trained. This means that BERT, based on the Transformer model architecture, applies its self-attention mechanism to learn information from a text from the left and right side during training, and consequently gains a deep understanding of the context. For example, the word fine can have two different meanings depending on the context (I feel fine today, She has fine blond hair). BERT considers the words surrounding the target word fine from the left and right side. However it comes at a cost: due to encoder-only architecture lacking a decoder, BERT can't be prompted and can't generate text, while bidirectional models in general do not work effectively without the right side, thus being difficult to prompt. As an illustrative example, if one wishes to use BERT to continue a sentence fragment "Today, I went to", then naively one would mask out all the tokens as "Today, I went to [MASK] [MASK] [MASK] ... [MASK] ." where the number of [MASK] is the length of the sentence one wishes to extend to. However, this constitutes a dataset shift, as during training, BERT has never seen sentences with that many tokens masked out. Consequently, its performance degrades. More sophisticated techniques allow text generation, but at a high computational cost.

Research Notes on Masked language model

Masked language model

>> Section 7

Bidirectional encoder representations from transformers (BERT) is a language model introduced in October 2018 by researchers at Google. It learns to represent text as a sequence of vectors using self-supervised learning. It uses the encoder-only transformer architecture. BERT dramatically improved the state of the art for large language models. As of 2020, BERT is a ubiquitous baseline in natural language processing (NLP) experiments. BERT is trained by masked token prediction and next sentence prediction. With this training, BERT learns contextual, latent representations of tokens in their context, similar to ELMo and GPT-2. It found applications for many natural language processing tasks, such as coreference resolution and polysemy resolution. It improved on ELMo and spawned the study of "BERTology", which attempts to interpret what is learned by BERT. BERT was originally implemented in the English language at two model sizes, BERTBASE (110 million parameters) and BERTLARGE (340 million parameters). Both were trained on the Toronto BookCorpus (800M words) and English Wikipedia (2,500M words). The weights were released on GitHub. On March 11, 2020, 24 smaller models were released, the smallest being BERTTINY with just 4 million parameters.

>> Section 8

Cost BERT was trained on the BookCorpus (800M words) and a filtered version of English Wikipedia (2,500M words) without lists, tables, and headers. Training BERTBASE on 4 cloud TPU (16 TPU chips total) took 4 days, at an estimated cost of 500 USD. Training BERTLARGE on 16 cloud TPU (64 TPU chips total) took 4 days.